

EECS 841

1

Q1

Machine Learning: Questions and Answers

Linear Regression

Question: Explain how the loss function is used to evaluate the performance of a linear regression model. Provide an example of a loss function.

Answer: The loss function quantifies the error between predicted and actual values. A common example is the Mean Squared Error (MSE) loss, which calculates the average of the squared differences between predicted and actual values:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Neural Networks

Question: Describe the process of backpropagation in neural networks and its purpose.

Answer: Backpropagation is used to update the parameters. It computes the gradient of the loss function with respect to each weight in the network, starting from the output layer and working backward to adjust the weights to minimize the loss.

Convolutional Neural Networks (CNN)

Question: Explain the purpose of convolutional layers and pooling layers in Convolutional Neural Networks (CNNs).

Answer: Convolutional layers extract features from the input data using filters. Pooling layers reduce the spatial size of the feature maps, decreasing the computational cost and helping to extract dominant features.

Recurrent Neural Networks (RNN)

Question: What is the key idea behind Recurrent Neural Networks (RNNs), and how do they process sequential data?

Answer: RNNs have an internal state (hidden state) that is updated as a sequence is processed. At each time step, the RNN takes an input and its previous hidden state to compute a new hidden state, which is then used to make a prediction. This allows RNNs to maintain information about past inputs in the sequence.

Computational Questions and Answers

1. Sigmoid Neuron Output

Question: Given input $x = [x_1, x_2, \dots, x_n]$, weights $w = [w_1, w_2, \dots, w_n]$, and bias b , compute the output $\hat{y}$ using the sigmoid function.

Answer:

$$z = w^T x + b$$
$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

2. Logistic Regression Loss Function

Question: What is the formula for the cross-entropy loss function in logistic regression?

Answer: Given predicted probability $\hat{y}$ and true label $y \in \{0, 1\}$:

$$L(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

3. Gradient Descent in Logistic Regression

Question: How are weights updated using gradient descent?

Answer: For weight w_j , learning rate α , and input x_j :

$$w_j := w_j - \alpha \cdot \frac{\partial L}{\partial w_j}$$
$$\frac{\partial L}{\partial w_j} = (\hat{y} - y)x_j$$

4. Backpropagation Weight Update

Question: What is the principle behind weight updates during backpropagation?

Answer:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$
$$w := w - \alpha \cdot \frac{\partial J}{\partial w}$$

5. Forward Propagation in a 2-layer MLP

Question: Given 3 inputs and 3 neurons in a hidden layer, what is the forward pass formula?

Answer: For each hidden neuron i :

$$r_i = b_i + \sum_j w_{ij} x_j, \quad a_i = \sigma(r_i)$$

Final output:

$$y = b + \sum_i c_i \cdot a_i$$

6. Mean Squared Error (MSE)

Question: Given predictions $\hat{y}_i$ and labels y_i , how is MSE computed?

Answer:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

7. Stochastic Gradient Descent (SGD)

Question: How do you update weights in SGD?

Answer:

$$w := w - \alpha \cdot \nabla_w J^{(i)}(w)$$

where $J^{(i)}$ is the loss from a single sample.

8. Dropout in Neural Networks

Question: What is dropout and how is it applied during training?

Answer: Randomly deactivate a portion of neurons (e.g., 30%) during each training iteration. At test time, all neurons are active and outputs are scaled.

9. L2 Regularization in Logistic Regression

Answer: It adds a penalty term to the loss:

$$L_{\text{reg}} = L + \lambda \|w\|_2^2$$

This discourages large weights, helping prevent overfitting.

Overfitting

Definition: Overfitting happens when a model learns not only the underlying patterns in the training data but also the noise or random fluctuations. As a result, the model performs very well on training data but poorly on new or unseen data.

Example:

Imagine a curve that passes through every training point exactly. While it fits the training data perfectly, it might be too complex and inaccurate for new data points.

Signs of Overfitting

- Low training error, but high validation/test error
- Poor real-world performance
- Very complex models with many parameters

How to Fix Overfitting

1. **Regularization:** Adds a penalty to large weights.

$$\text{L2 (Ridge): } \lambda \sum w^2 \quad \text{L1 (Lasso): } \lambda \sum |w|$$

2. **Simplify the Model:** Use fewer layers, neurons, or features.
3. **More Training Data:** Increase generalization with more or augmented data.
4. **Early Stopping:** Stop training when validation loss increases.
5. **Dropout:** Randomly drop neurons during training.
6. **Cross-Validation:** Use multiple data splits to test generalization.